

Java

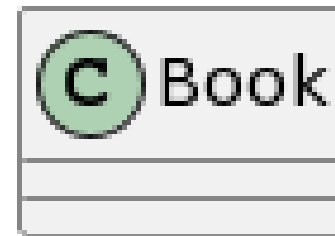
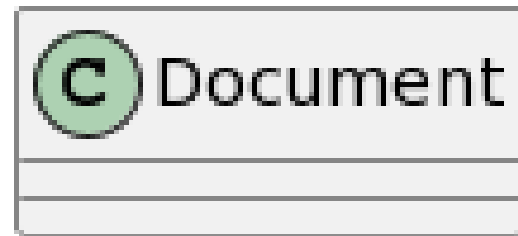
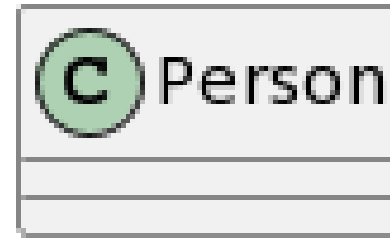
Orientado a Objetos

Método principal de la aplicación

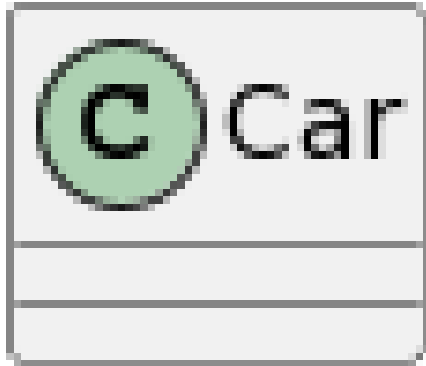
- La palabra clave `public static void main` es el medio por el cual se crea un método principal dentro de la aplicación Java. Es el método central del programa y llama a todos los demás. No puede devolver valores y acepta parámetros para el procesamiento complejo de la línea de comandos.
- No se puede modificar la cabecera del método.

```
public class Main {  
    public static void main(String[] args)  
    {  
    }  
}
```

Ejemplo: Codificar

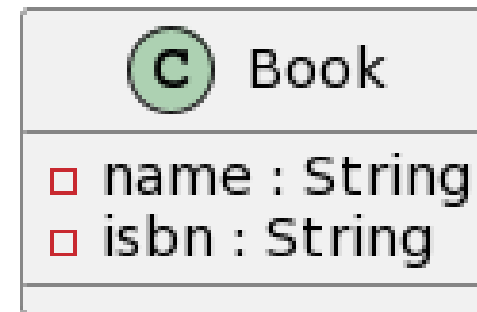
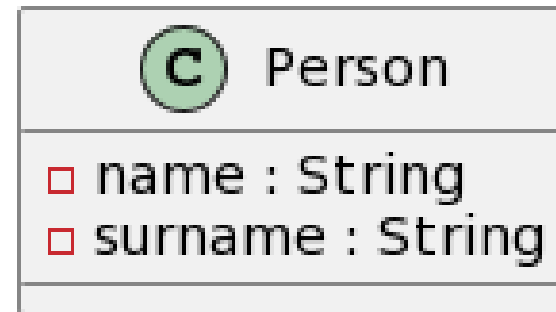
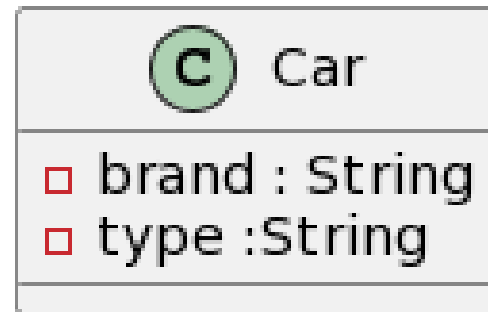


Forma más simple de diagrama de clases

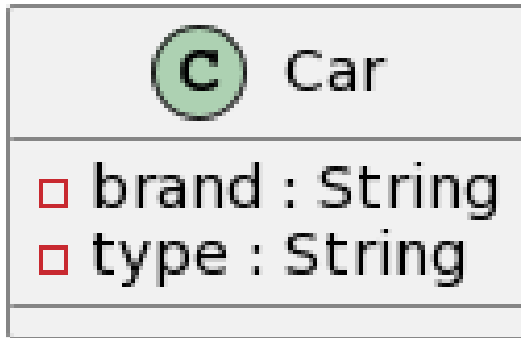


```
public class Car {  
}
```

Ejercicio: Codificar



Clase con atributos



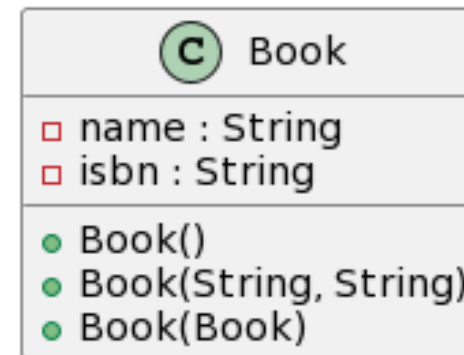
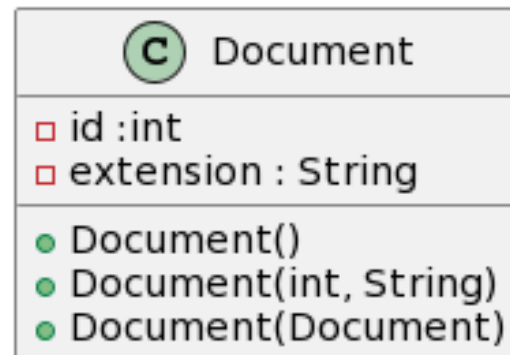
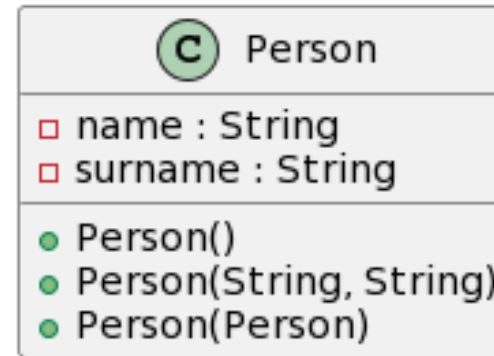
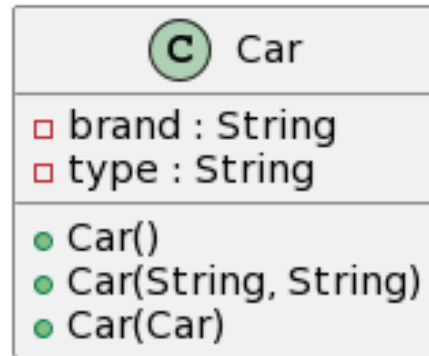
```
public class Car {  
    private String brand;  
    private String type;  
  
    public Car(String brand, String type) {  
        this.brand = brand;  
        this.type = type;  
    }  
    public String getBrand() {  
        return this.brand;  
    }  
    public void setBrand(String brand) {  
        This.brand = brand;  
    }  
    public String getType() {  
        return this.type;  
    }  
    public void setType(String type) {  
        this.type = type;  
    }  
}
```

Uso de clases en Main

```
public class Main {  
    public static void main(String[] args)  
    {  
        Car car1 = new Car("Mazda", "6");  
        System.out.println(car1.getBrand());  
    }  
}
```

```
public class Main {  
    public static void main(String[] args)  
    {  
        Person person1 = new Person("Rober",  
        "Martin");  
        System.out.println(person1.getName())  
        ;  
    }  
}
```

Constructores



Constructores

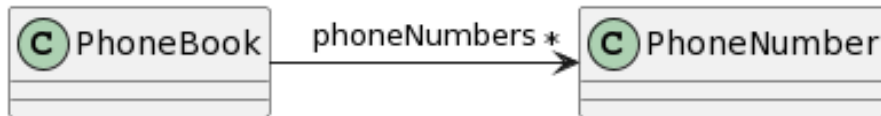
```
public Car(){  
}  
  
public Car(String brand, String type) {  
    this.brand = brand;  
    this.type = type;  
}  
  
public Car(Car car) {  
    this.brand = car.getBrand();  
    this.type = car.getType();  
}
```

```
public Person(){  
}  
  
public Person(String name, String  
surname) {  
    this.name = name;  
    this.surname = surname;  
}  
  
public Person(Person person) {  
    this.name = person.getName();  
    this.surname = person.getSurname();  
}
```

Asociación



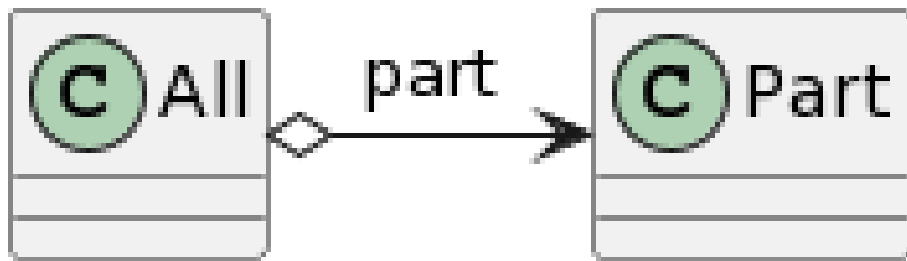
```
public class Phone {  
    private Button buttons[15];  
}  
public class Button {}
```



```
public class PhoneBook {  
    private List<PhoneNumber> phoneNumbers;  
}  
public class PhoneNumber {}
```

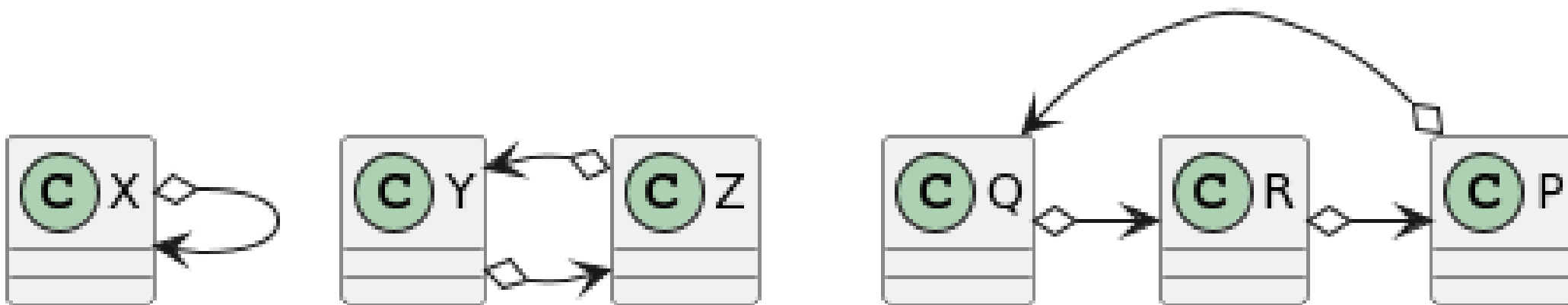
Ponemos las asociaciones en horizontal

Agregación

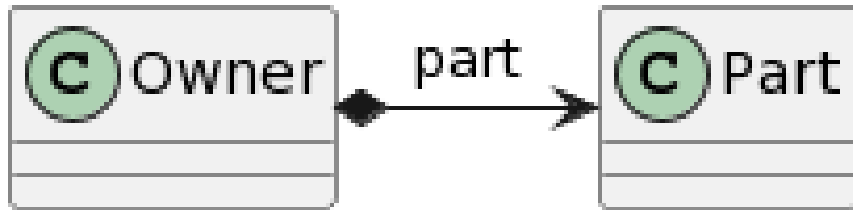


```
public class All {  
    private Part part;  
}
```

Agregaciones ilegales

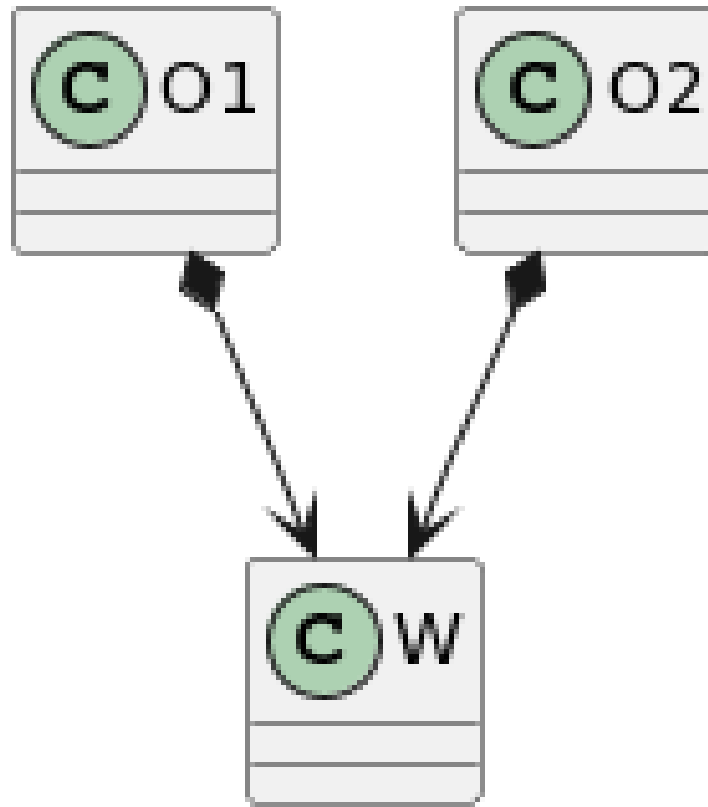


Composición

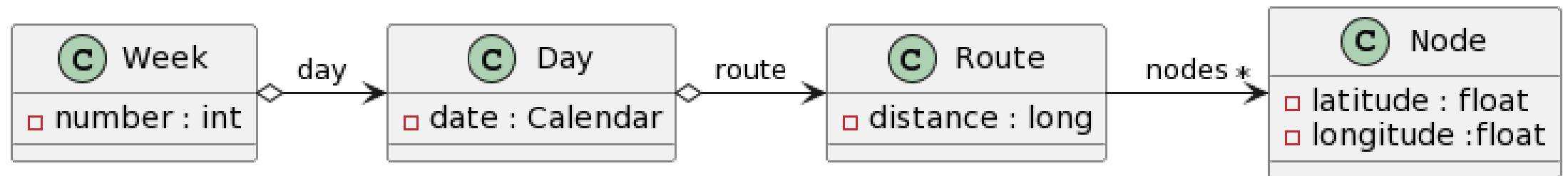


```
public class Owner {  
    private Part part;  
}
```

Composición ilegal



Ejercicio: Codificar



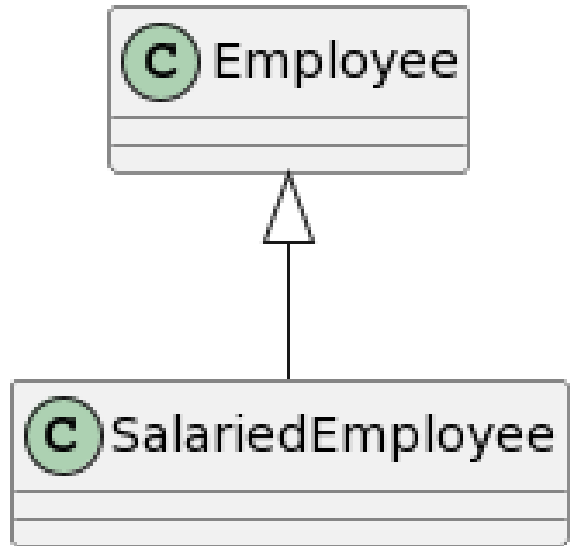
```
public class Week {  
    private int number;  
    private Day day;  
}
```

```
public class Day {  
    private Calendar date;  
    private Route route;  
}
```

```
public class route {  
    private long distance;  
    private List<Node> nodes;  
}
```

```
public class Node {  
    private float latitude;  
    private float longitude;  
}
```

Herencia

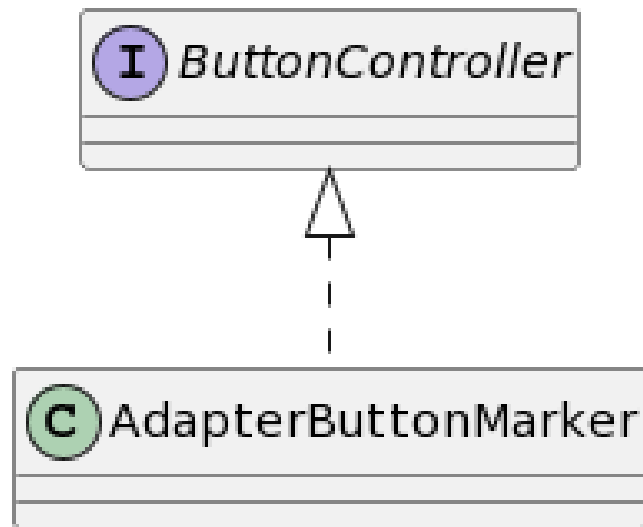


```
public class Employee {  
}
```

```
public class SalariedEmployee  
    extends Employee {  
}
```

Ponemos las herencias en vertical

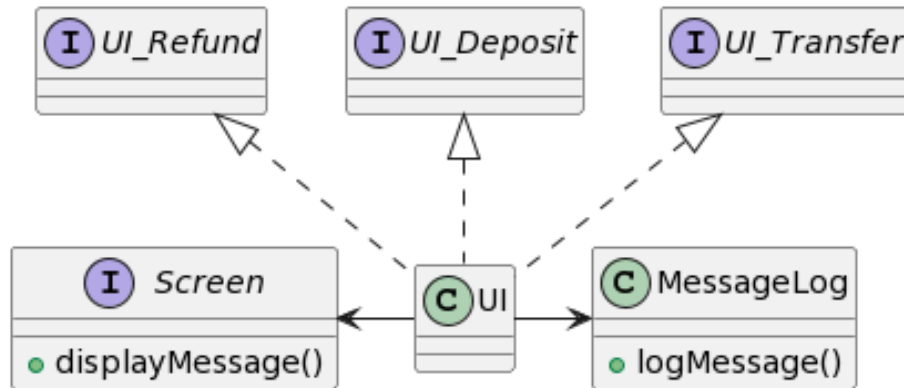
Interfaz (Herencia de métodos)



```
interface ButtonController {  
}
```

```
public class AdapterButtonMarker implements  
    ButtonController {  
}
```

Ejemplo



```
public class UI implements UI_Refund,  
                           UI_Deposit, UI_Transfer {  
    private Screen itsScreen;  
    private MessageLog itsMessageLog;
```

```
    [...]
```

```
    public void displayMessage(String message) {  
        itsMessageLog.logMessage(message);  
        itsScreen.displayMessage(message);  
    }
```

```
}
```

Ejercicio: Codificar

I <i>Constructive</i>
• <code>construct()</code> • <code>construct(Solution)</code>

A <i>Animal</i>
□ <code>name : String</code> □ <code>age : int</code>
• <code>roar()</code>

I <i>Improvement</i>
• <code>improve()</code> • <code>improve(Solution)</code>

A <i>Vehicle</i>
□ <code>brand : String</code> □ <code>type : String</code>
• <code>beep()</code>

Ejemplo

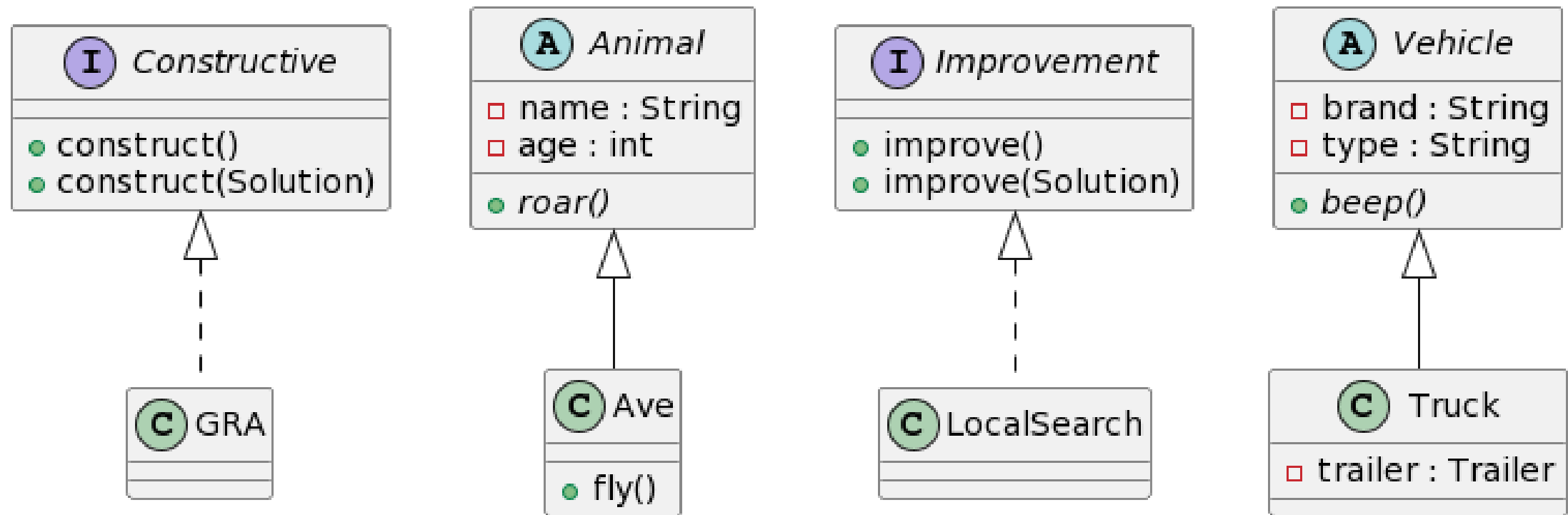
```
public interface Constructive {  
    public void construct();  
    public void construct(Solution solution);  
}
```

```
public abstract class Animal {  
    private String name;  
    private int age;  
    abstract void roar();  
}
```

```
public interface Improvement {  
    public void improve();  
    public void improve(Solution solution);  
}
```

```
public abstract class Vehicle {  
    private String Brand;  
    private String type;  
    abstract void beep();  
}
```

Ejercicio: Codificar



Ejemplo

```
public interface Constructive {  
    public void construct();  
    public void construct(Solution solution);  
}
```

```
public class GRA implements Constructive {  
    public void construct() {}  
    public void construct(Solution solution) {}  
}
```

```
public abstract Animal {  
    private String name;  
    private int age;  
    abstract void roar();  
}
```

```
public class Ave extends Animal {  
    public void roar {}  
}
```

```
public interface Improvement {  
    public void improve();  
    public void improve(Solution solution);  
}
```

```
public class LocalSearch implements Improvement {  
    public void improve() {}  
    public void improve(Solution solution) {}  
}
```

```
public abstract class Vehicle {  
    private String Brand;  
    private String type;  
    abstract void beep();  
}
```

```
public class Truck extends Vehicle {  
    private Trailer trailer;  
}
```

Ejemplo

```
public class Main {  
    public static void main(String[] args) {  
        Animal chicken = new Ave("Clotilda", 1);  
        chicken.roar();  
        chicken.fly();//acceso ilegal, no ve el método  
        Ave ave = new Ave("Claudio", 2);  
        ave.roar();  
        ave.fly();  
        Vehicle trailer = new Truck("Man", "TGX", new  
            Trailer());  
        trailer.beep();  
        Truck truck = new Truck("Volvo", "FH", new  
            trailer());  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        GRA constructor1 = new GRA();  
        Constructive constructor2 = new GRA();  
        constructor1.construct();  
        constructor2.construct(new Solution());  
        LocalSearch ls1 = new LocalSearch();  
        Improvement ls2 = new LocalSearch();  
        ls1.improve();  
        ls2.improve(new Solution());  
    }  
}
```